Fundamentals of Communication Systems

Lab 2 **CNSCC-160**

In this lab you will learn:

- Different colour spaces
- Bit plane decomposition
- Image quantization
- Image compression
- Fidelity measure

Background Knowledge

A **digital image** is composed of ***pixels*** which can be thought of as small dots on the screen.

Image Types

I. Intensity image (Greyscale image)

This form represents an image as a matrix where every element has a value corresponding to how bright/dark the pixel is at the corresponding position.

II. Binary image

This image format also stores an image as a matrix but can only colour a pixel black or white (and nothing in between). 0 – is for black and a 1 – is for white.

IV. RGB image

This format represents an image with three matrices of sizes matching the image format. Each matrix corresponds to one of the colours red, green or blue and instructs how much of each of these colours a certain pixel should use.

Task 1: Read a colour image (lena.tif or image of your choice) and display the Red, Green and Blue components. Your output should be

Colour Image	**Red Image**
Green Image	**Blue Image**

Hint:

```
% Code to display RGB image
clc;            % clear the command window
close all;          % closes all figures
% Open a standard MATLAB image.

rgbImage = imread('lena.tif');  % read the image

% Display the original image.
subplot(2, 2, 1);
imshow(rgbImage);
title('RGB image');

% Extract out the individual channels.
RImage = rgbImage(:,:,1);
.
.
.
.
```

If you like you can assign different colourmaps to R, G, B components.

Task 2: Read a colour image (lena.tif or image of your choice), convert it YCbCr image and display the individual components.

Hint:

```
% Code to display YCbCr image
clc;            % clear the command window
close all;          % closes all figures
% Open a standard MATLAB image.
rgbImage = imread('lena.tif');  % read the image

% Convert to YCbCr color space
YCbCr = rgb2ycbcr(rgbImage);

% Display the original image.
subplot(2, 2, 1);
imshow (YCbCr, [ ]);
title(' YCbCr  image');



% Extract out the individual channels.
YImage = YCbCr(:,:,1);
.
% Display the Y  image.

.
.
```

.

Task 3: Read a colour image (lena.tif or image of your choice) convert it to grey and then extract and display the bit planes.

Hint:

```
% Code to display bit planes images
clc;            % clear the command window
close all;          % closes all figures
% Open a standard MATLAB image.
rgbImage = imread('lena.tif');  % read the image

% Convert to gray image
Img = rgb2gray(rgbImage);

Get bit at specified position
b1 = double(bitget(Img,1)); % bit plane 0
b2 = double(bitget(Img,2)); % bit plane 1
.
.
.

b7 = double(bitget(Img,8)); % bit plane 7
.
.
.
% Display the bit planes image.
figure,
imshow(Img), title('Original:');
figure,
subplot(4,2,1)
imshow(b1), title('Bit Plane: 1');
subplot(4,2,2)
.
.
.
.
```

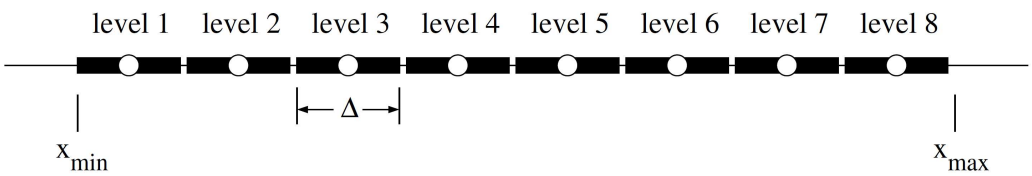
Task 4: Quantisation

Quantization is the most elementary form of lossy data compression, while also forming a fundamental part of more advanced lossy compression schemes such as Jpeg. When a number x is **quantized to L levels**, we mean that its value is replaced by (or quantized to) the nearest member of a set of **L quantisation levels**. Here, we consider **uniform quantisation**.

For the uniform quantization used here:

- We define a **quantiser range** defined by values $x_{\min}$ and $x_{\max}$
- We divide this range into L equally sized segments, each with size $\triangle = \frac{x_{\max} - x_{\min}}{L}$.
- We place the quantization level for a given segment in the middle of that segment.

The quantiser is illustrated with the figure shown below, which shows $L = 8$ segments of width $\triangle = (x_{\max} - x_{\min})/8$ as thick lines and the corresponding levels within each segment as circles.



The value of $\triangle = (x_{\max} - x_{\min})/L$ is usually small otherwise it increases the maximum possible error introduced by the quantiser.

In this Task, we will experiment with direct quantization as an image compression mechanism. Download the function quantise.m.

- **Use 'quantise.m' to quantise the "lena.tif" (or image of your choice) image using 128 levels, 64 levels, 32 levels, 16levels, 8 levels and 4 levels.**
- **Assume $x_{\min} = 0$ and $x_{\max} = 255$.**
    - **E.g. [outim,delta]=quantise(input_image,level,0,255);**

  Hint:

  ```
  % Open a standard MATLAB image.
  rgbImage = imread('lena.tif');  % read the image

  % Convert to gray image
  Img = rgb2gray(rgbImage);
  [outim,delta]=quantise(Img,128,0,255);
  ..
  ..
  ..
  ```

- **Display and include in your report the six resulting quantized images along with the original using a subplot.**
- **Describe the effects of the quantization in these plots and plot a graph of MSE vs the number of bits. MSE can be calculated as**
    - **MSE = immse(outim,input_image)**

Task 5:

In order to complete this task, use the provided jpeg_compression_cycle function to compress the "lena.tif" (or image of your choice) image, i.e.

outim= jpeg_compression_cycle (input_image, quality)
repeat the experiment for different value of quality (i.e. 10, 20, 30, 40, 50, 60, 80, 90).
Hint:

```
% Open a standard MATLAB image.
Img = imread('lena.tif');  % read the image

% Convert to gray image
outim= jpeg_compression_cycle  (Img,20);
..
…
..
```

- **Display the seven resulting compressed images along with the original using subplot.**
- **Describe the effects of the compression in these plots and plot a graph of MSE vs compression quality. MSE can be calculated as**
    - **MSE = immse(outim, Img)**

Appendix 1

Name	Convert input to
• gray2ind	- intensity image to index image (uint8 or uint16)
• im2bw	- image to binary (logical type)
• im2double	- image to double precision (double)
• im2uint8	- image to 8-bit unsigned integers (uint8)
• im2uint16	- image to 16-bit unsigned integers (uint16)
• ind2gray	- indexed image to intensity image (double)
• mat2gray	- matrix to intensity image (double)
• rgb2gray	- RGB image to grayscale (double)
• rgb2ind	- RGB image to indexed image
• mat2gray	- double (in range [0,1])